Materiale fornito come supporto per lo studio del corso di SWAM tenuto presso UNIFI. Alcune delle figure o dei loghi presenti possono essere tratte da pubblicazioni o siti web e quindi soggette a copyright. Pertanto la distribuzione di questo materiale è lecita se effettuata a fini di insegnamento o di ricerca scientifica il cui utilizzo sia per finalità illustrative o di studio e per fini non commerciali.

A.A. 2021/2022

Angular Framework - Part II

Ing. Jacopo Parri, Ing. Samuele Sampietro

STLab - Software Technologies Lab

Materiale di supporto al corso di
Software Architectures and Methodologies - Laboratory

UNIVERSITÀ DEGLI STUDI FIRENZE

Software Technologies Lab - Dipartimento di Ingegneria dell'Informazione (UNIFI)

Programma [argomenti generali]

- Architettura di Angular
  - panoramica di base
  - i pilastri
- Anatomia di un Progetto Angular
  - strumenti utili
  - generare un progetto "from scratch"
  - anatomia del codice
- **Approfondimenti fondamentali**
  - lavorare con i Servizi e la DI
  - il concetto di Routing
  - stilizzare la UI tramite un "design system"

Materiale fornito come supporto per lo studio del corso di SWAM tenuto presso UNIFI. Alcune delle figure o dei loghi presenti possono essere tratte da pubblicazioni o siti web e quindi soggette a copyright. Pertanto la distribuzione di questo materiale è lecita se effettuata a fini di insegnamento o di ricerca scientifica il cui utilizzo sia per finalità illustrative o di studio e per fini non commerciali.

Servizi e Dependency Injection

Angular Framework

Ing. Jacopo Parri, Ing. Samuele Sampietro

STLab - Software Technologies Lab

Materiale di supporto al corso di
Software Architectures and Methodologies - Laboratory

UNIVERSITÀ DEGLI STUDI FIRENZE

Software Technologies Lab - Dipartimento di Ingegneria dell'Informazione (UNIFI)

Introduzione [1/2]

Fino a questo punto del corso abbiamo utilizzato qualche Componente in modo "improprio":

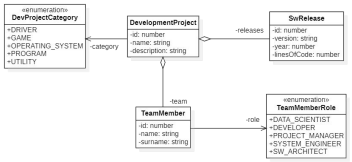
- infatti, un Componente **non dovrebbe mantenere e gestire direttamente dati**
- ma **dovrebbe avere la sola responsabilità di presentare i dati**
  - offrendo una pagina di presentazione adeguata (i.e. template)
  - implementando una logica di business che supporti le azioni della pagina (i.e. component controller)

Introduzione [2/2]

Per ovviare alle problematiche enunciate in precedenza si devono adottare i **Servizi**:

- un Servizio è un valido **intermediario per la condivisione di dati** tra classi
  - quindi, è uno strumento da adottare anche tra Componenti
- un Servizio **non necessita della conoscenza dei richiedenti** le sue funzionalità
  - è indipendente dai Client
- un Servizio **può essere facilmente "iniettato"** dove richiesto
  - si rende fondamentale il meccanismo di Dependency Injection di Angular

Il nostro "primo" Servizio



Abbiamo dunque bisogno di:

- ristrutturare i Componenti del nostro **scenario di esempio**
  - *DevelopmentProjectListComponent*
  - *DevelopmentProjectDetailComponent*
  - *DevelopmentProjectEditorComponent*
- introdurre un **Servizio**
  - *DevelopmentProjectService*

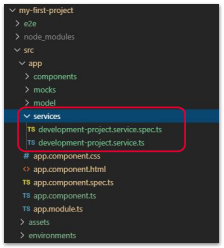
Bisogna **spostare** alcune **responsabilità** !

Generiamo il Servizio

Comando:

```
ng generate service development-project
```

```
CREATE src/app/services/development-project.service.spec.ts (394 bytes)
CREATE src/app/services/development-project.service.ts (147 bytes)
```



DevelopmentProjectService [1/2]

Angular CLI ha generato un Servizio con una implementazione di base:

- è presente sulla classe il decoratore `@Injectable()`
  - indica che la classe partecipa attivamente nel processo di Dependency Injection
  - il metadato `providedIn: 'root'` indica che può essere iniettata ovunque e che Angular creerà una singola istanza condivisa del Servizio

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class DevelopmentProjectService {

  constructor() { }
}
```

development-project.service.ts

Questo Servizio **NON** fa niente !

Rendiamo il Servizio utile !

DevelopmentProjectService [2/2]

Per **liberare** il Componente "lista" della responsabilità di erogare la lista di Progetti direttamente, basta delegare al nuovo Servizio:

- è sufficiente far restituire al Servizio la lista di mock `DEV_PROJECTS`
- rimuovere tale riferimento da *DevelopmentProjectListComponent*

```
import { Injectable } from '@angular/core';
import { DevelopmentProject } from '../model/development-project';
import { DEV_PROJECTS } from '../mocks/development-projects-mock';

@Injectable({
  providedIn: 'root'
})
export class DevelopmentProjectService {

  constructor() { }

  getDevProjects(): DevelopmentProject[] {
    return DEV_PROJECTS;
  }
}
```

development-project.service.ts

```
@Component({
  selector: 'app-development-project-list',
  templateUrl: './development-project-list.component.html',
  styleUrls: ['./development-project-list.component.css']
})
export class DevelopmentProjectListComponent implements OnInit {
  developmentProjects: DevelopmentProject[] = DEV_PROJECTS;
  selectedProject: DevelopmentProject = null;
  selectedProjectToEdit: DevelopmentProject = null;
  . . .
```

development-project-list.component.ts

Semplice !

Adattare il Componente "lista"

development-project-list.component.ts

```
import { DevelopmentProjectService } from '../../services/development-project.service';
import { DevelopmentProject } from '../../model/development-project';
import { Component, OnInit } from '@angular/core';

@Component({
  selector: 'app-development-project-list',
  templateUrl: './development-project-list.component.html',
  styleUrls: ['./development-project-list.component.css']
})
export class DevelopmentProjectListComponent implements OnInit {
  developmentProjects: DevelopmentProject[];
  selectedProject: DevelopmentProject = null;
  selectedProjectToEdit: DevelopmentProject = null;

  constructor(private devProjectService: DevelopmentProjectService) {
    console.log( 'Constructor of DevelopmentProjectListComponent has been invoked!' );
  }

  ngOnInit() {
    this.getDevProjects();
    console.log( 'Hello, component named DevelopmentProjectListComponent has been initialized!' );
  }

  getDevProjects(): void {
    this.developmentProjects = this.devProjectService.getDevProjects();
  }

  selectProject(p: DevelopmentProject): void {
    this.selectedProjectToEdit = null;
    this.selectedProject = p;
  }

  selectProjectToEdit(p: DevelopmentProject): void {
    this.selectedProject = null;
    this.selectedProjectToEdit = p;
  }

  clearSelectedProjects(): void {
    this.selectedProject = null;
    this.selectedProjectToEdit = null;
  }
}
```

DI nel costruttore

Cosa è cambiato ?

Eseguendo l'applicazione NON è cambiato niente:

- e, in più, sappiamo di "star leggendo" il mock dei Progetti da un Servizio !

Attualmente, il metodo che invochiamo:

```
this.developmentProjects = this.devProjectService.getDevProjects();
```

- ha un **comportamento sincrono**
- questo aspetto **non è realistico** rispetto a quando consumeremo i dati da un server remoto (via HTTP)

Rendiamo asincrono il Servizio

Ci serviremo di un **Observable**, tramite la libreria **RxJS**:

- la funzione `of(...)`
  - ritorna un oggetto `Observable<DevelopmentProject[]>` che emette un valore singolo con la lista di Progetti mock

```
import { Injectable } from '@angular/core';
import { DevelopmentProject } from '../model/development-project';
import { DEV_PROJECTS } from '../mocks/development-projects-mock';
import { Observable, of } from 'rxjs';

@Injectable({
  providedIn: 'root'
})
export class DevelopmentProjectService {

  constructor() { }

  getDevProjects(): Observable<DevelopmentProject[]> {
    return of(DEV_PROJECTS);
  }
}
```

development-project.service.ts

Gli **Observable** si possono considerare come *"lazy push collections"* di valori multipli, emessi da sorgenti alle quali ci si può "sottoscrivere" per riceverne gli aggiornamenti

Consumare il Servizio asincrono

```
ngOnInit() {
  this.getDevProjects();
  console.log( 'Hello, component named DevelopmentProjectListComponent has been initialized!' );
}

getDevProjects(): void {
  this.devProjectService.getDevProjects()
    .subscribe(retrievedProjects => this.developmentProjects = retrievedProjects);
}
```

development-project-list.component.ts

Si noti l'uso del metodo **`subscribe()`**:

- tramite la sua invocazione il Client del Servizio viene posto in attesa per una "futura" ricezione dell'oggetto `Observable` che emetterà la lista di `DevelopmentProject[]`
- al momento della ricezione, il metodo `subscribe()` passerà la lista alla sua callback

Approfondiamo il consumare Servizi

Come accennato, solitamente i frontend di una Architettura RESTful devono consumare servizi REST:

- questo avviene tramite **protocollo HTTP**
  - che mette a disposizione i suoi verbi (GET, POST, PUT, PATCH, DELETE)

Dobbiamo approfondire il modo di **compiere richieste HTTP** che dovranno essere compiute da un Servizio Angular !

Il client HTTP di Angular

Il framework mette a disposizione un Servizio "speciale" chiamato **`HttpClient`**:

- si trova all'interno del modulo **`HttpClientModule`**
  - il quale deve essere importato nel solito *AppModule*

```
1  import { BrowserModule } from '@angular/platform-browser';
2  import { NgModule } from '@angular/core';
3  import { FormsModule } from '@angular/forms';
4  import { HttpClientModule } from '@angular/common/http';
5
6  import { AppComponent } from './app.component';
7  import { DevelopmentProjectListComponent } from './components/develo
8  import { DevelopmentProjectDetailComponent } from './components/deve
9  import { DevelopmentProjectEditorComponent } from './components/deve
10
11 @NgModule({
12   declarations: [
13     AppComponent,
14     DevelopmentProjectListComponent,
15     DevelopmentProjectDetailComponent,
16     DevelopmentProjectEditorComponent
17   ],
18   exports: [
19     AppComponent
20   ],
21   imports: [
22     BrowserModule,
23     FormsModule,
24     HttpClientModule
25   ],
26   providers: [],
27   bootstrap: [AppComponent]
28 })
29 export class AppModule { }
```

app.module.ts

Data server remoto "simulato"

Quando in **via di sviluppo**, non si dispone di un backend che esponga servizi REST, ma si deve procedere con i codici del frontend, è conveniente servirsi di un server simulato:

- una soluzione valida (in generale) è quella offerta da **JSON Server**
  - https://github.com/typicode/json-server

Get a full fake REST API with **zero coding** in **less than 30 seconds** (seriously)

- mentre, la soluzione offerta da Angular è **In-Memory Web API**
  - https://github.com/angular/in-memory-web-api
  - è uno strumento citato nel Tutorial *"Tour of Heroes"*

Aggiungere una dipendenza tramite Angular CLI

Si può aggiungere una libreria esterna al Progetto tramite questo comando:

```
ng add <collection> [options]
```

Aggiunge la dipendenza del package npm specificato con il parametro <collection>, configurando il workspace e aggiornando il file **package.json**

Aggiungiamo la dipendenza In-Memory Web API

Eseguiamo il comando:

```
ng add angular-in-memory-web-api
```

La dipendenza viene aggiunta al nostro package.json e viene installato il package:

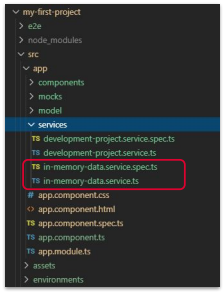
```
"dependencies": {
  "@angular/animations": "~8.2.11",
  "@angular/common": "~8.2.11",
  "@angular/compiler": "~8.2.11",
  "@angular/core": "~8.2.11",
  "@angular/forms": "~8.2.11",
  "@angular/platform-browser": "~8.2.11",
  "@angular/platform-browser-dynamic": "~8.2.11",
  "@angular/router": "~8.2.11",
  "angular-in-memory-web-api": "^0.9.0",
  "normalize.css": "^8.0.1",
  "rxjs": "~6.4.0",
  "tslib": "^1.10.0",
  "zone.js": "~0.9.1"
},
```


Generiamo il Servizio per In-Memory Web API

Comando:

```
ng generate service in-memory-data
```

```
CREATE src/app/services/in-memory-data.service.spec.ts (365 bytes)
CREATE src/app/services/in-memory-data.service.ts (141 bytes)
```



Importare In-Memory Web API dentro AppModule

Bisogna assicurarsi di avere importato dentro alla configurazione della app:

- `HttpClientInMemoryWebApiModule`
  - questo modulo intercetterà le richieste HTTP e produrrà risposte simulate
- `InMemoryDataService`
  - è il Servizio che fornirà i dati e popolerà il DB simulato
  - va registrato sul modulo precedente tramite `forRoot()`

```
import { BrowserModule } from '@angular/platform-browser';
import { NgModule } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { HttpClientModule } from '@angular/common/http';

import { HttpClientInMemoryWebApiModule } from 'angular-in-memory-web-api';
import { InMemoryDataService } from './services/in-memory-data.service';

import { AppComponent } from './app.component';
import { DevelopmentProjectListComponent } from './components/development-projec
import { DevelopmentProjectDetailComponent } from './components/development-proj
import { DevelopmentProjectEditorComponent } from './components/development-proj

@NgModule({
  declarations: [
    AppComponent,
    DevelopmentProjectListComponent,
    DevelopmentProjectDetailComponent,
    DevelopmentProjectEditorComponent
  ],
  exports: [
    AppComponent
  ],
  imports: [
    BrowserModule,
    FormsModule,
    HttpClientModule,
    HttpClientInMemoryWebApiModule.forRoot(
      InMemoryDataService, { dataEncapsulation: false }
    )
  ],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

app.module.ts

Il core del DB simulato

Il nostro Servizio implementa `InMemoryDbService` che obbliga a definire il metodo:

- `createDb()`
  - ritorna la lista di Progetti mock e la colloca In-Memory
- in questo momento si sta anche definendo la URI dell'endpoint simulato
  - `api/dev_projects_collection`

```
import { DevelopmentProject } from 'src/app/model/development-project';
import { DEV_PROJECTS } from './../mocks/development-projects-mock';
import { Injectable } from '@angular/core';
import { InMemoryDbService } from 'angular-in-memory-web-api';

@Injectable({
  providedIn: 'root'
})
export class InMemoryDataService implements InMemoryDbService {

  constructor() { }

  createDb() {
    const dev_projects_collection = DEV_PROJECTS;
    return {dev_projects_collection};
  }

  genId(devProjects: DevelopmentProject[]): number {
    return devProjects.length > 0 ? Math.max(...devProjects.map(devProj => devProj.id)) + 1 : 11;
  }
}
```

in-memory-data.service.ts

Usiamo `HttpClient` dentro a DevelopmentProjectService

In questo caso, in modo del tutto naturale, ci serviremo di una istanza del Servizio `HttpClient`:

- la DI è risolta nel **costruttore** e associata ad un attributo di classe privato
- viene invocato il metodo `get<DevelopmentProject[]>(this.entityURI)`

```
import { Injectable } from '@angular/core';
import { DevelopmentProject } from '../model/development-project';
import { Observable, of } from 'rxjs';
import { HttpClient } from '@angular/common/http';

@Injectable({
  providedIn: 'root'
})
export class DevelopmentProjectService {
  private entityURI = 'api/dev_projects_collection';

  constructor(private http: HttpClient) { }

  getDevProjects(): Observable<DevelopmentProject[]> {
    return this.http.get<DevelopmentProject[]>(this.entityURI);
  }
}
```

development-project.service.ts

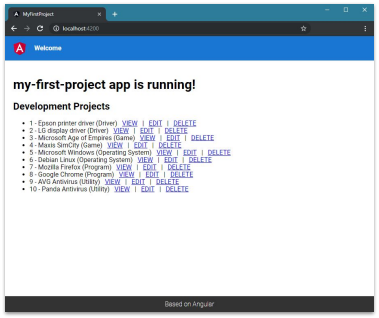
Si sta realizzando uno scenario **"service-in-service"**

Preview di test con server simulato

Sempre andando su:

- `http://localhost:4200/`

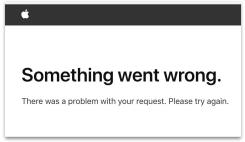
TUTTO OK !



Observable error handling [1/3]

Quando si effettuano delle richieste HTTP verso server remoti, qualcosa potrebbe "andare storto":

- si deve quindi **gestire gli errori in modo adeguato**
  - tendenzialmente "catturandoli" e illudendo l'utente finale che nel frontend sia tutto OK



Observable error handling [2/3]

La libreria **RxJS**, dalla quale abbiamo tratto il tipo `Observable`, mette a disposizione degli sviluppatori:

- la funzione `pipe()`
  - che permette di compiere una sequenza di operazioni sul risultato in emissione
  - in input richiede le funzioni (i.e. gli operatori) da applicare in ordine prima di terminare
- l'operatore `catchError()`
  - che permette di catturare un eventuale errore propagato da un osservabile "fallito"
  - in input richiede una funzione per la gestione dell'errore

Observable error handling [3/3]

Si noti che:

- l'operatore `catchError()`
  - riceve in input
    la funzione templatizzata
    `handleError()`
    che stampa un log nella console
    e ritorna un "empty result"

```
import { Injectable } from '@angular/core';
import { DevelopmentProject } from '../model/development-project';
import { Observable, of } from 'rxjs';
import { catchError } from 'rxjs/operators';
import { HttpClient } from '@angular/common/http';

@Injectable({
  providedIn: 'root'
})
export class DevelopmentProjectService {
  private entityURI = 'api/dev_projects_collection';

  constructor(private http: HttpClient) { }

  getDevProjects(): Observable<DevelopmentProject[]> {
    return this.http.get<DevelopmentProject[]>(this.entityURI)
      .pipe(
        catchError(this.handleError<DevelopmentProject[]>('getDevProjects', []))
      );
  }

  private handleError<T> (operation = 'operation', result?: T) {
    return (error: any): Observable<T> => {
      console.error(error);

      console.log(`${operation} failed: ${error.message}`);

      return of(result as T);
    };
  }
}
```

development-project.service.ts

Cosa abbiamo imparato finora ?

Tramite l'uso coordinato di Servizi e Dependency Injection è possibile dotare qualsiasi Componente di un intermediario che espone metodi utili (ad esempio) a interfacciarsi verso un Endpoint REST:

- si può affermare che, per come li stiamo implementando, **i Servizi che adottano `HttpClient` assumono lo stesso ruolo che in un backend hanno i Data Access Object (DAO)**
  - i DAO si interfacciano al DBMS
  - i Servizi che stiamo descrivendo, si interfacciano ai servizi REST tramite verbi HTTP

Implementiamo meglio il "nostro" **DevelopmentProjectService**

DevelopmentProjectService [CRUD via HTTP verbs]

Dobbiamo prevedere almeno questi 5 metodi:

- `getDevProjects()`
  - Retrieve all
  - GET `api/dev_projects_collection`
- `getDevProject(id: number)`
  - Retrieve one
  - GET `api/dev_projects_collection/{id}`
- `updateDevProject(p: DevelopmentProject)`
  - Update
  - PUT `api/dev_projects_collection`
- `deleteDevProject(id: number)`
  - Delete
  - DELETE `api/dev_projects_collection/{id}`
- `addDevProject(name: string)`
  - Create
  - POST `api/dev_projects_collection`

getDevProjects()

GET `api/dev_projects_collection`

L'abbiamo già visto:

- implementa il caso d'uso di recupero di una lista di `DevelopmentProject`
- invoca
  - **`http.get()`**

```
private entityURI = 'api/dev_projects_collection';
private httpOptions = {
  headers: new HttpHeaders({ 'Content-Type': 'application/json' })
};
```

```
getDevProjects(): Observable<DevelopmentProject[]> {
  return this.http.get<DevelopmentProject[]>(this.entityURI)
    .pipe(
      catchError(this.handleError<DevelopmentProject[]>('getDevProjects', []))
    );
}
```

development-project.service.ts

getDevProject(id: number)

GET `api/dev_projects_collection/{id}`

In questo caso:

- implementa il caso d'uso di recupero di un `DevelopmentProject`
  - in base al parametro **`id`**
- invoca
  - **`http.get()`**

```
private entityURI = 'api/dev_projects_collection';
private httpOptions = {
  headers: new HttpHeaders({ 'Content-Type': 'application/json' })
};
```

```
getDevProject(id: number): Observable<DevelopmentProject> {
  const url = `${this.entityURI}/${id}`;

  return this.http.get<DevelopmentProject>(url)
    .pipe(
      catchError(this.handleError<DevelopmentProject>('getDevProject'))
    );
}
```

development-project.service.ts

updateDevProject(p: DevelopmentProject)

PUT `api/dev_projects_collection`

In questo caso:

- implementa il caso d'uso di aggiornamento di un `DevelopmentProject`
  - in base al parametro **`p`**
- invoca
  - **`http.put()`**

```
private entityURI = 'api/dev_projects_collection';
private httpOptions = {
  headers: new HttpHeaders({ 'Content-Type': 'application/json' })
};
```

```
updateDevProject(p: DevelopmentProject): Observable<DevelopmentProject> {
  return this.http.put<DevelopmentProject>(this.entityURI, p, this.httpOptions)
    .pipe(
      catchError(this.handleError<DevelopmentProject>('updateDevProject'))
    );
}
```

development-project.service.ts

deleteDevProject(id: number)

DELETE `api/dev_projects_collection/{id}`

In questo caso:

- implementa il caso d'uso di cancellazione di un `DevelopmentProject`
  - in base al parametro **`id`**
- invoca
  - **`http.delete()`**

```
private entityURI = 'api/dev_projects_collection';
private httpOptions = {
  headers: new HttpHeaders({ 'Content-Type': 'application/json' })
};
```

```
deleteDevProject(id: number): Observable<DevelopmentProject> {
  const url = `${this.entityURI}/${id}`;

  return this.http.delete<DevelopmentProject>(url, this.httpOptions)
    .pipe(
      catchError(this.handleError<DevelopmentProject>('deleteDevProject'))
    );
}
```

development-project.service.ts

`addDevProject(name: string)`

POST `api/dev_projects_collection`

```
private entityURI = 'api/dev_projects_collection';
private httpOptions = {
  headers: new HttpHeaders({ 'Content-Type': 'application/json' })
};
```

In questo caso:

- implementa il caso d'uso di creazione di un `DevelopmentProject`
  - inizializzato con un **`name`** e **`id`** autogenerato
- invoca
  - **`http.post()`**

```
addDevProject(name: string): Observable<DevelopmentProject> {
  const p = new DevelopmentProject(null, name);

  return this.http.post<DevelopmentProject>(this.entityURI, p, this.httpOptions)
    .pipe(
      catchError(this.handleError<DevelopmentProject>('addDevProject'))
    );
}
```

development-project.service.ts

Integriamo anche i vari Componenti

Per vedere all'opera il Servizio che implementa i 5 metodi bisogna aggiornare anche:

- il Componente "lista"
- il Componente "dettaglio"
- il Componente "editore"

Andiamo a vedere come cambiano


```
1  import { DevelopmentProjectService } from './../../services/development-project.service';
2  import { DevelopmentProject } from './../../model/development-project';
3  import { Component, OnInit } from '@angular/core';
4
5  @Component({
6    selector: 'app-development-project-list',
7    templateUrl: './development-project-list.component.html',
8    styleUrls: ['./development-project-list.component.css']
9  })
10 export class DevelopmentProjectListComponent implements OnInit {
11   developmentProjects: DevelopmentProject[];
12   selectedProjectId: number = 0;
13   selectedProjectToEditId: number = 0;
14   tmpName: string;
15
16   constructor(private devProjectService: DevelopmentProjectService) {
17     console.log( 'Constructor of DevelopmentProjectListComponent has been invoked!' );
18   }
19
20   ngOnInit() {
21     this.getDevProjects();
22     console.log( 'Hello, component named DevelopmentProjectListComponent has been initialized!' );
23   }
24
25   getDevProjects(): void {
26     this.devProjectService.getDevProjects()
27       .subscribe(retrievedProjects => this.developmentProjects = retrievedProjects);
28   }
29
30   deleteDevProject(id: number): void {
31     this.devProjectService.deleteDevProject(id)
32       .subscribe(
33         () => {
34           console.log('Development Project ' + id + ' deleted!');
35           this.clearSelectedProjects();
36         }
37       );
38   }
39
40   createDevProject(): void {
41     this.devProjectService.addDevProject(this.tmpName)
42       .subscribe(
43         newProject => {
44           console.log('Development Project ' + newProject.id + ' created!');
45           this.tmpName = "";
46           this.clearSelectedProjects();
47         }
48       );
49   }
50
51   selectProject(pId: number): void {
52     this.selectedProjectToEditId = 0;
53     this.selectedProjectId = pId;
54   }
55
56   selectProjectToEdit(pId: number): void {
57     this.selectedProjectId = 0;
58     this.selectedProjectToEditId = pId;
59   }
60
61   clearSelectedProjects(): void {
62     this.selectedProjectId = 0;
63     this.selectedProjectToEditId = 0;
64     this.getDevProjects();
65   }
66 }
```

development-project-list.component.ts


```
1  <div>
2      <div style="float: right;">
3          <input type="text" [(ngModel)]="tmpName" placeholder="New Dev Project name"/>
4
5          <button type="button"
6              (click)="createDevProject()">Add Dev Project</button>
7      </div>
8
9
10     <h2>Development Projects</h2>
11
12     <ul>
13         <li *ngFor="let p of developmentProjects">
14             {{ p.id }} - {{ p.name }} ({{ p.category }})
15              
16             <a (click)="selectProject(p.id)" style="cursor: pointer;">VIEW</a>
17              
18             |
19              
20             <a (click)="selectProjectToEdit(p.id)" style="cursor: pointer;">EDIT</a>
21              
22             |
23              
24             <a (click)="deleteDevProject(p.id)" style="cursor: pointer;">DELETE</a>
25         </li>
26     </ul>
27 </div>
28
29 <app-development-project-detail [devProjectId]="selectedProjectId" (clearEvent)="clearSelectedProjects($event)"></app-development-project-detail>
30
31 <app-development-project-editor [devProjectId]="selectedProjectToEditId" (clearEvent)="clearSelectedProjects($event)"></app-development-project-editor>
```

development-project-list.component.html

```
import { DevelopmentProject } from './../../model/development-project';
import { Component, OnInit, Input, EventEmitter, Output, OnChanges, SimpleChanges } from '@angular/core';
import { DevelopmentProjectService } from 'src/app/services/development-project.service';

@Component({
  selector: 'app-development-project-detail',
  templateUrl: './development-project-detail.component.html',
  styleUrls: ['./development-project-detail.component.css']
})
export class DevelopmentProjectDetailComponent implements OnInit, OnChanges {
  @Input() devProjectId: number;
  @Output() clearEvent = new EventEmitter<void>();
  devProject: DevelopmentProject;

  constructor(private devProjectService: DevelopmentProjectService) { }

  ngOnChanges(changes: SimpleChanges) {
    if(this.devProjectId)
      this.retrieveDevProject(this.devProjectId);
  }

  ngOnInit() {
  }

  retrieveDevProject(id: number): void {
    this.devProjectService.getDevProject(id)
      .subscribe(retrievedProject => this.devProject = retrievedProject);
  }

  clearProject() {
    this.devProjectId = 0;
    this.devProject = null;
    this.clearEvent.emit();
  }
}
```

development-project-detail.component.ts

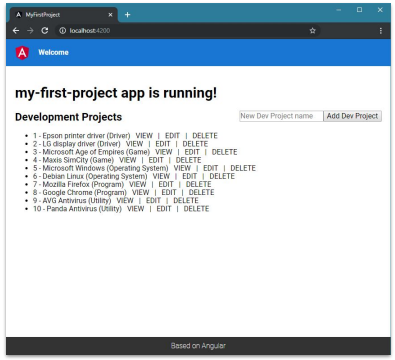

```
import { Component, OnInit, Input, EventEmitter, Output, OnChanges, SimpleChanges } from '@angular/core';
import { DevelopmentProject } from 'src/app/model/development-project';
import { DevProjectCategory } from 'src/app/model/dev-project-category';
import { DevelopmentProjectService } from 'src/app/services/development-project.service';

@Component({
  selector: 'app-development-project-editor',
  templateUrl: './development-project-editor.component.html',
  styleUrls: ['./development-project-editor.component.css']
})
export class DevelopmentProjectEditorComponent implements OnInit, OnChanges {
  @Input() devProjectId: number;
  @Output() clearEvent = new EventEmitter<void>();
  devProject: DevelopmentProject;

  projectCategoriesEnum = DevProjectCategory;
  projectCategoriesKeys: string[];

  constructor(private devProjectService: DevelopmentProjectService) {
    this.projectCategoriesKeys = Object.keys(this.projectCategoriesEnum);
  }

  ngOnChanges(changes: SimpleChanges) {
    if(this.devProjectId)
      this.retrieveDevProject(this.devProjectId);
  }

  ngOnInit() {
  }

  retrieveDevProject(id: number): void {
    this.devProjectService.getDevProject(id)
      .subscribe(retrievedProject => this.devProject = retrievedProject);
  }

  updateDevProject(): void {
    this.devProjectService.updateDevProject(this.devProject)
      .subscribe(() => this.clearProject());
  }

  clearProject() {
    this.devProjectId = 0;
    this.devProject = null;
    this.clearEvent.emit();
  }
}
```

development-project-editor.component.ts

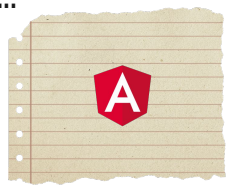
Preview finale della app

Si noti:

- il campo di `<input>` vicino al pulsante che permette l'aggiunta di un nuovo Progetto di sviluppo



Prossimamente...



Routing in Angular

Materiale fornito come supporto per lo studio del corso di SWAM tenuto presso UNIFI. Alcune delle figure o dei loghi presenti possono essere tratte da pubblicazioni o siti web e quindi soggette a copyright. Pertanto la distribuzione di questo materiale è lecita se effettuata a fini di insegnamento o di ricerca scientifica il cui utilizzo sia per finalità illustrative o di studio e per fini non commerciali.

Angular Framework

Ing. Jacopo Parri, Ing. Samuele Sampietro

STLab - Software Technologies Lab

Materiale di supporto al corso di
Software Architectures and Methodologies - Laboratory

UNIVERSITÀ DEGLI STUDI FIRENZE

Programma [argomenti generali]

- Architettura di Angular
  - panoramica di base
  - i pilastri
- Anatomia di un Progetto Angular
  - strumenti utili
  - generare un progetto "from scratch"
  - anatomia del codice
- **Approfondimenti fondamentali**
  - lavorare con i Servizi e la DI
  - il concetto di Routing
  - stilizzare la UI tramite un "design system"

Materiale fornito come supporto per lo studio del corso di SWAM tenuto presso UNIFI. Alcune delle figure o dei loghi presenti possono essere tratte da pubblicazioni o siti web e quindi soggette a copyright. Pertanto la distribuzione di questo materiale è lecita se effettuata a fini di insegnamento o di ricerca scientifica il cui utilizzo sia per finalità illustrative o di studio e per fini non commerciali.

Il Routing

Angular Framework

Ing. Jacopo Parri, Ing. Samuele Sampietro

STLab - Software Technologies Lab

Materiale di supporto al corso di
Software Architectures and Methodologies - Laboratory

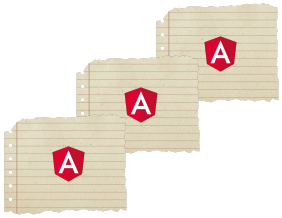
UNIVERSITÀ DEGLI STUDI FIRENZE

Introduzione

L'applicazione DEMO che abbiamo progettato e implementato **non prevede pagine dedicate** per i vari Componenti:

- **tutto si realizza all'interno di una singola pagina HTML**
  - quella di *DevelopmentProjectListComponent*
- tale Componente "lista" si limita a incapsulare i 2 TAG riferiti ai selettori degli altri Componenti
  - *DevelopmentProjectDetailComponent*
  - *DevelopmentProjectEditorComponent*

Introduzione



Si rende necessario estendere la app:

- realizzando **pagine separate** per i 3 Componenti
  - ognuna con una propria URL
- abilitando la **navigazione tra pagine**, mantenendo una buona *user experience*
  - servendosi di **Angular Router**

Angular Router

Ogni app implementata con Angular si basa sul package `@angular/router` per il routing:

- il Router messo a disposizione permette di **interpretare una URL** (del browser) **come un'istruzione** per navigare verso una nuova pagina client-side
  - una app è pur sempre una single-page application
- il Router può **passare parametri opzionali** al Componente associato ad una vista
- il Router **ammette diverse modalità di attivazione**
  - navigazione guidata da link (e.g. ancore)
  - navigazione imperativa (e.g. click su un `<button>`)
  - storicizzata (i.e. pulsanti *back* & *forward* del browser)

Come si abilita il Router ?

Per prima cosa si deve **generare** un nuovo modulo dentro alla app:

- tale modulo prenderà il nome di **AppRoutingModule**
  - in questo modo, si separeranno gli aspetti di *navigazione* da quelli di *business logic*
- dovrà contenere
  - le **configurazioni di navigazione** per l'applicazione
  - le **associazioni "URI interne" <====> "Componente"**
  - le definizioni di eventuali **guardie di accesso alle viste**

Generare Moduli tramite Angular CLI

Si possono generare nuovi Moduli tramite questo comando:

```
ng generate module <name> [options]
```

Crea un Modulo dal nome specificato con il parametro <name>

Conviene spostarsi all'interno della cartella scelta come destinazione per lanciare il comando

Proviamo a generare il Modulo di “routing”

Se lanciamo il comando:

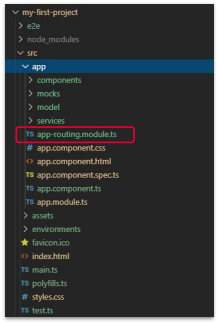
```
ng generate module app-routing --module app --flat
```

Nella vostra console si vedrà:

```
CREATE src/app/app-routing.module.ts (196 bytes)
UPDATE src/app/app.module.ts (1327 bytes)
```


Infatti...

Un nuovo file è stato all’interno di `/src/app/`



Ed è anche stato automaticamente importato dentro `app.module.ts`

app-routing.module.ts [1/2]

File preposto a contenere la configurazione del Modulo con le configurazioni di routing della app.

Questa versione è quella di base:

- presenta il decoratore di classe `@NgModule()`
- importa `CommonModule`
  - ossia un modulo built-in di Angular che esporta un insieme di direttive generali per qualsiasi modulo
  - dato che per il routing non ci interessa, sparirà dalla sua implementazione

```
import { NgModule } from '@angular/core';
import { CommonModule } from '@angular/common';

@NgModule({
  imports: [
    CommonModule
  ],
  declarations: []
})
export class AppRoutingModule { }
```

Adesso procediamo con implementare il **modulo**

app-routing.module.ts [2/2]

L’implementazione *“intermedia”* prevede:

- l’importazione di `RouterModule`
  - quello citato da `@angular/router`
- l’uso del metodo statico `RouterModule.forRoot()`
  - il quale crea il modulo di routing per Angular
  - lo configura a livello radice con le rotte previste
    - `appRoutes: Routes`
  - lo inizializza con le direttive di navigazione
    - `RouterLink`
    - `RouterLinkActive`
    - `RouterLinkWithHref`
    - `RouterOutlet`

```
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';

const appRoutes: Routes = [];

@NgModule({
  declarations: [],
  imports: [
    RouterModule.forRoot(appRoutes)
  ],
  exports: [
    RouterModule
  ]
})
export class AppRoutingModule { }
```

Attualmente **NON** vi sono **“rotte”**

Accorgimenti: `<base>`

Prima di definire realmente le "rotte" della app bisogna prendere degli accorgimenti:

- verifichiamo che all'interno del file `src/index.html` compaia il TAG `<base>`
- in particolare...

```html
<!doctype html>
<html lang="en">

<head>
  <meta charset="utf-8">
  <title>MyFirstProject</title>
  <base href="/">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link rel="icon" type="image/x-icon" href="favicon.ico">
</head>

<body>
  <app-root></app-root>
</body>

</html>
```

Fissa la **base** per il routing

Accorgimenti: `<router-outlet>`

Inoltre, si deve aggiungere il TAG `<router-outlet>`:

- all'interno del file `src/app/app.component.html`
- nel punto in cui si desidera che il Router **carichi "dinamicamente" in visualizzazione** i Componenti sotto routing
  - si intuisce che si dovranno, prima o poi, associare i Componenti alle route

Se ricordate, prima, abbiamo citato la direttiva `RouterOutlet`

Questo TAG speciale è messo a disposizione dalla direttiva !

app.component.html

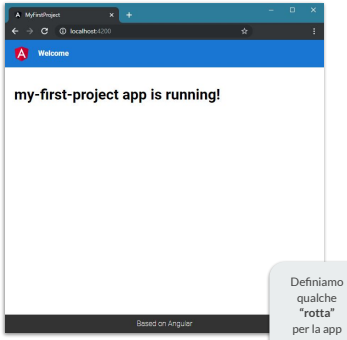
```html
<div id="main-skeleton">
  <!-- Toolbar -->
  <div class="toolbar" role="banner">
    <img
      width="40"
      alt="Angular Logo"
      src="
    />
    <span>Welcome</span>
  </div>

  <!-- Main content -->
  <div class="content" role="main">
    <h1 id="main-title">{{ title }} app is running!</h1>

    <router-outlet></router-outlet>
  </div>

  <!-- Footer -->
  <footer>
    Based on Angular
  </footer>
</div>
```

In cosa differisce dalla versione del file alla quale vi siete abituati ?

Preview senza "route definition"

Per come è stato configurato **AppRoutingModule**:

- è corretto **NON** visualizzare niente!
  - oltre il `title`



Definiamo qualche **"rotta"** per la app

Rotte principali

app-routing.module.ts

```
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';

import { DevelopmentProjectListComponent } from './components/development-project-list/development-project-list.component';
import { DevelopmentProjectDetailComponent } from './components/development-project-detail/development-project-detail.component';
import { DevelopmentProjectEditorComponent } from './components/development-project-editor/development-project-editor.component';

const appRoutes: Routes = [
  { path: 'dev-projects',  component: DevelopmentProjectListComponent },
  { path: 'dev-projects/:id', component: DevelopmentProjectDetailComponent },
  { path: 'dev-projects/edit/:id', component: DevelopmentProjectEditorComponent },
  { path: '',   redirectTo: '/dev-projects', pathMatch: 'full' }
];

@NgModule({
  declarations: [],
  imports: [
    RouterModule.forRoot(appRoutes)
  ],
  exports: [
    RouterModule
  ]
})
export class AppRoutingModule { }
```


Ristrutturazione dei Componenti

Per rendere funzionante la navigazione della app è necessario fare anche ulteriori modifiche ai codici di:

- **DevelopmentProjectListComponent**
  - `development-project-list.html`
  - `development-project-list.ts`
- **DevelopmentProjectDetailComponent**
  - `development-project-detail.html`
  - `development-project-detail.ts`
- **DevelopmentProjectEditorComponent**
  - `development-project-editor.html`
  - `development-project-editor.ts`

Vediamo le **modifiche necessarie** per ogni file

Works in progress [1/6]

Notare che, rispetto alla versione precedente:

- sono stati rimossi i TAG associati ai Componenti "dettaglio" e "editore"
- sono stati modificati i link "azione", introducendo la direttiva `routerLink`

```
<div>
    <div style="float: right;">
        <input type="text" [(ngModel)]="tmpName" placeholder="New Dev Project name"/>

        <button type="button"
            (click)="createDevProject()">Add Dev Project</button>
    </div>


    <h2>Development Projects</h2>

    <ul>
        <li *ngFor="let p of developmentProjects">
            {{ p.id }} - {{ p.name }} ({{ p.category }})
             
            <a routerLink="/dev-projects/{{ p.id }}" style="cursor: pointer;">VIEW</a>
             
            |
             
            <a routerLink="/dev-projects/edit/{{ p.id }}" style="cursor: pointer;">EDIT</a>
             
            |
             
            <a (click)="deleteDevProject(p.id)" style="cursor: pointer;">DELETE</a>
        </li>
    </ul>
</div>
```

development-project-list.component.html

Works in progress [2/6]

Notare che, rispetto alla versione precedente:

- sono stati rimossi i metodi
  - `selectProject()`
  - `selectProjectToEdit()`
  - `clearSelectedProjects()`

```
import { DevelopmentProjectService } from '../../../services/development-project.service';
import { DevelopmentProject } from '../../../model/development-project';
import { Component, OnInit } from '@angular/core';

@Component({
  selector: 'app-development-project-list',
  templateUrl: './development-project-list.component.html',
  styleUrls: ['./development-project-list.component.css']
})
export class DevelopmentProjectListComponent implements OnInit {
  developmentProjects: DevelopmentProject[];
  tmpName: string;

  constructor(private devProjectService: DevelopmentProjectService) {
    console.log('Constructor of DevelopmentProjectListComponent has been invoked!' );
  }

  ngOnInit() {
    this.getDevProjects();
    console.log('Hello, component named DevelopmentProjectListComponent has been initialized!' );
  }

  getDevProjects(): void {
    this.devProjectService.getDevProjects()
      .subscribe(retrievedProjects => this.developmentProjects = retrievedProjects);
  }

  deleteDevProject(id: number): void {
    this.devProjectService.deleteDevProject(id)
      .subscribe(
        () => {
          console.log('Development Project ' + id + ' deleted!');
          this.getDevProjects();
        }
      );
  }

  createDevProject(): void {
    this.devProjectService.addDevProject(this.tmpName)
      .subscribe(
        newProject => {
          console.log('Development Project ' + newProject.id + ' created!');
          this.tmpName = "";
          this.getDevProjects();
        }
      );
  }
}
```

development-project-list.component.ts

Works in progress [3/6]

Notare che, rispetto alla versione precedente:

- è stato riscritto il `<button>` con azione di chiusura; rendendolo di fatto un pulsante "Indietro"

```
1   <div *ngIf="devProjectId != 0 && devProject">
2     <h2>{{ devProject.name | uppercase }} Details</h2>
3
4     <button type="button"
5             routerLink="/dev-projects/"
6             style="float: right;">Close</button>
7
8     <div>
9         <b>Id: </b>{{ devProject.id }}
10         | 
11        <b>category: </b> {{ devProject.category }}
12         | 
13        <b>releases: </b> {{ devProject.releases.length }}
14        <span *ngIf="devProject.releases.length > 0">
15          [1st - {{ devProject.firstReleaseYear }}] [Last - {{ devProject.lastReleaseYear }}]
16        </span>
17    </div>
18
19    <hr/>
20
21    <div>
22        <b>description: </b>{{ devProject.description }}
23    </div>
24
25    <hr/>
26
27    <div>
28        <b>team members: </b>
29
30        <span *ngIf="devProject.team && devProject.team.length > 0">
31            {{devProject.team.length}}
32            <ul>
33                <li *ngFor="let m of devProject.team">
34                    {{ m.name }} {{ m.surname }} ({{ m.role }})
35                </li>
36            </ul>
37        </span>
38
39        <span *ngIf="!devProject.team || devProject.team.length == 0">0</span>
40    </div>
41  </div>
```

development-project-detail.component.html

Works in progress [4/6]

Notare che, rispetto alla versione precedente:

- sono stati rimossi i decoratori `@Input` e `@Output`
- il recupero tramite Servizio del `DevelopmentProject` è stato spostato dentro `ngOnInit()`
- è stato rimosso il metodo `clearProject()`

```
1   import { DevelopmentProject } from './../../model/development-project';
2   import { Component, OnInit } from '@angular/core';
3   import { DevelopmentProjectService } from 'src/app/services/development-project.service';
4   import { ActivatedRoute } from '@angular/router';
5
6   @Component({
7     selector: 'app-development-project-detail',
8     templateUrl: './development-project-detail.component.html',
9     styleUrls: ['./development-project-detail.component.css']
10  })
11  export class DevelopmentProjectDetailComponent implements OnInit {
12    devProjectId: number;
13    devProject: DevelopmentProject;
14
15    constructor(
16      private route: ActivatedRoute,
17      private devProjectService: DevelopmentProjectService) { }
18
19    ngOnInit() {
20      this.devProjectId = +this.route.snapshot.paramMap.get('id');
21
22      console.log('Detail ' + this.devProjectId);
23
24      if(this.devProjectId)
25        this.retrieveDevProject(this.devProjectId);
26    }
27
28    retrieveDevProject(id: number): void {
29      this.devProjectService.getDevProject(id)
30        .subscribe(retrievedProject => this.devProject = retrievedProject);
31    }
32  }
```

Estrazione del parametro `id` dalla URI della `route`

development-project-detail.component.ts

Works in progress [5/6]

Notare che, rispetto alla versione precedente:

- è stato riscritto il `<button>` con azione di chiusura
- è stato aggiunto un `<button>` con azione di salvataggio

```
1   <div *ngIf="devProjectId != 0 && devProject">
2     <h2>{{ devProject.name | uppercase }} Edit</h2>
3
4     <button type="button"
5             routerLink="/dev-projects/"
6             style="float: right;">Close</button>
7
8     <form #editProjectForm="ngForm">
9         <button type="button"
10                [disabled]="editProjectForm.invalid"
11                (click)="updateDevProject()"
12                style="float: right;">Save</button>
13
14        <div>
15            <b>Id: </b>{{ devProject.id }}
16        </div>
17
18        <hr/>
19
20        <div>
21            <b>name: </b>
22            <input type="text" name="name" [(ngModel)]="devProject.name" placeholder="name"
23                   minlength="10" required #name="ngModel"/>
24
25            <div *ngIf="name.invalid && (name.dirty || name.touched)" class="alert alert-danger">
26                <div *ngIf="name.errors.required">
27                    The field "name" is required.
28                </div>
29
30                <div *ngIf="name.errors.minlength">
31                    Min lenght of "name" should be 10 characters.
32                </div>
33            </div>
34        </div>
35        <div>
36            <b>category: </b>
37            <select name="category" [(ngModel)]="devProject.category">
38                <option *ngFor="let key of projectCategoriesKeys" [ngValue]="projectCategoriesEnum[key]">
39                    {{ projectCategoriesEnum[key] }}
40                </option>
41            </select>
42        </div>
43        <div>
44            <b>description: </b>
45            <textarea name="description" [(ngModel)]="devProject.description" placeholder="description"></textarea>
46        </div>
47    </form>
48  </div>
```

development-project-editor.component.html

Works in progress [6/6]

Notare che, rispetto alla versione precedente:

- sono stati rimossi i decoratori `@Input` e `@Output`
- il recupero tramite Servizio del `DevelopmentProject` è stato spostato dentro `ngOnInit()`
- è stato rimosso il metodo `clearProject()`

```
1   import { Component, OnInit } from '@angular/core';
2   import { DevelopmentProject } from 'src/app/model/development-project';
3   import { DevProjectCategory } from 'src/app/model/dev-project-category';
4   import { DevelopmentProjectService } from 'src/app/services/development-project.service';
5   import { ActivatedRoute, Router } from '@angular/router';
6
7   @Component({
8     selector: 'app-development-project-editor',
9     templateUrl: './development-project-editor.component.html',
10    styleUrls: ['./development-project-editor.component.css']
11  })
12  export class DevelopmentProjectEditorComponent implements OnInit {
13    devProjectId: number;
14    devProject: DevelopmentProject;
15
16    projectCategoriesEnum = DevProjectCategory;
17    projectCategoriesKeys: string[];
18
19    constructor(
20      private route: ActivatedRoute,
21      private router: Router,
22      private devProjectService: DevelopmentProjectService
23    ) {
24      this.projectCategoriesKeys = Object.keys(this.projectCategoriesEnum);
25    }
26
27    ngOnInit() {
28      this.devProjectId = +this.route.snapshot.paramMap.get('id');
29
30      console.log('Edit ' + this.devProjectId);
31
32      if(this.devProjectId)
33        this.retrieveDevProject(this.devProjectId);
34    }
35
36    retrieveDevProject(id: number): void {
37      this.devProjectService.getDevProject(id)
38        .subscribe(retrievedProject => this.devProject = retrievedProject);
39    }
40
41    updateDevProject(): void {
42      this.devProjectService.updateDevProject(this.devProject)
43        .subscribe(() => this.goBack());
44    }
45
46    goBack() {
47      this.router.navigate(['/dev-projects']);
48    }
49  }
```

Routing dinamico per via programmatica

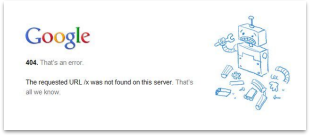
tramite `router`

development-project-editor.component.ts

Una pagina di errore

Per la nostra app è importante disporre anche di una pagina di errore:

- che sia **raggiungibile in automatico**
  - ogni volta che un client tenti di contattare una URI inesistente
- sarebbe la classica pagina che riporta la scritta **404 Page Not Found**



Proviamo a generare il Componente "404"

Se lanciamo il comando:

```
ng generate component page-not-found
```

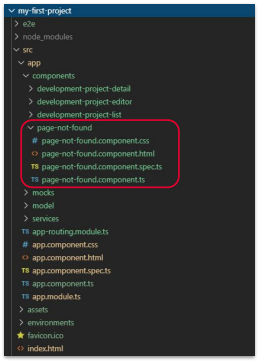
Nella vostra console si vedrà:

```
CREATE src/app/components/page-not-found/page-not-found.component.html (29 bytes)
CREATE src/app/components/page-not-found/page-not-found.component.spec.ts (672 bytes)
CREATE src/app/components/page-not-found/page-not-found.component.ts (299 bytes)
CREATE src/app/components/page-not-found/page-not-found.component.css (0 bytes)
UPDATE src/app/app.module.ts (1450 bytes)
```


Infatti...

Sono state generati i 4 file del componente all'interno di `/src/app/components/page-not-found`:

- `page-not-found.component.css`
- `page-not-found.component.html`
- `page-not-found.component.spec.ts`
- `page-not-found.component.ts`



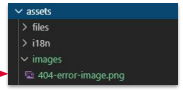
PageNotFoundComponent

```
import { Component, OnInit } from '@angular/core';

@Component({
  selector: 'app-page-not-found',
  templateUrl: './page-not-found.component.html',
  styleUrls: ['./page-not-found.component.css']
})
export class PageNotFoundComponent implements OnInit {

  constructor() { }

  ngOnInit() {
  }
```

`page-not-found.component.ts`

```
<h2>Page Not Found</h2>
<img src="assets/images/404-error-image.png" style="width: 100%;">
```

`page-not-found.component.html`



Aggiorniamo le rotte

Notare che si deve registrare una nuova rotta tra quelle previste per `RouterModule`:

- con la sintassi `path: '**'`
  - che indica la rotta di tipo wildcard

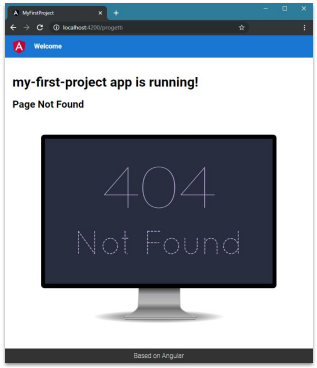
```
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';

import { DevelopmentProjectListComponent } from './components/development-project-list/developme
import { DevelopmentProjectDetailComponent } from './components/development-project-detail/devel
import { DevelopmentProjectEditorComponent } from './components/development-project-editor/devel
import { PageNotFoundComponent } from './components/page-not-found/page-not-found.component';

const appRoutes: Routes = [
  { path: 'dev-projects', component: DevelopmentProjectListComponent },
  { path: 'dev-projects/:id', component: DevelopmentProjectDetailComponent },
  { path: 'dev-projects/edit/:id', component: DevelopmentProjectEditorComponent },
  { path: '',   redirectTo: '/dev-projects', pathMatch: 'full' },
  { path: '**', component: PageNotFoundComponent }
];

@NgModule({
  declarations: [],
  imports: [
    RouterModule.forRoot(appRoutes)
  ],
  exports: [
    RouterModule
  ]
})
export class AppRoutingModule { }
```

app-routing.module.ts

Preview "page-not-found"

Per come è stato configurato **AppRoutingModule**, navigando su una URI inesistente:

- e.g. `http://localhost:4200/progetti`
- si visualizza il nuovo Componente
  - *PageNotFoundComponent*



Regole di matching per le route

Il Router di Angular adotta per le rotte una **First-match wins strategy**:

- ciò significa che i percorsi più specifici dovrebbero essere collocati **al di sopra** di percorsi meno specifici
- la rotta **wildcard** deve stare in ultima posizione
  - permettendo al Router di valutare prima ogni altra rotta

```
const appRoutes: Routes = [
  { path: 'home',  component: HomeComponent },
  { path: 'page-1/:param', component: Page1Component },
  { path: 'page-2', component: Page2Component },
  { path: '',   redirectTo: '/home', pathMatch: 'full' },
  { path: '**', component: PageNotFoundComponent }
];
```